

Visitor Management System (VMS)

Project Architecture & Documentation Overview

This document provides a comprehensive overview of the Visitor Management System (VMS) application. The application is structured into a React-based frontend and a Flask-based backend, utilizing modern security standards, role-based access control, and license verification parameters.

1. Project Directory Structure

vms/	
-- vms_backend/	Flask-based Backend Service
-- routes/	API routers (Authentication, Visitors, Requests, Admin, Security)
-- auth_routes.py	Authentication & user registration
-- visitor_routes.py	Public registration and status lookup
-- request_routes.py	Host approval and checkout request actions
-- security_routes.py	Security gate logs, badge printing, vehicle logs
-- admin_routes.py	Admin panels, settings management, site and user CRUD
-- reports_routes.py	Excel/PDF report exports
-- utils/	Helpers (License verification, password hashing, etc.)
-- license_verifier.py	Checks subscription status and keys
-- password_utils.py	Hashing algorithms and migrations
-- app.py	Application entrypoint, middleware, & CORS setup
-- db.py	MySQL Connection management
-- init_db.py	Database schemas initialization
-- requirements.txt	Python dependencies
-- vms_frontend/	React / Vite Single Page Application
-- src/	
-- app/	Store configuration & API Client
-- apiClient.js	Axios configuration with JWT injection
-- store.js	Redux central store config
-- routes/	App routing & Route Guards
-- AppRoutes.jsx	Client-side router & path registrations
-- ProtectedRoute.jsx	Role checks & authorization guards
-- layouts/	Dashboard wraps (Admin, Host, Security layouts)
-- components/	Reusable UI components (Buttons, inputs, tables)
-- pages/	Application views
-- LandingPage/	Public kiosk visitor dashboard
-- VisitorRegistration/	Entry forms for check-in
-- SecurityDashboard/	Gate logs & badge management
-- admin/	User administration & system settings
-- host/	Meeting invitations & approvals
-- SuperAdmin/	SaaS subscription & global metrics
-- LicenseRestricted/	Subscription renewal prompts

2. Frontend Architecture & Usability

The frontend is a Single Page Application (SPA) built using **Vite**, **React Router DOM**, **Tailwind CSS**, and **Redux Toolkit**.

- **main.jsx**: Entrypoint that mounts the React app onto the HTML DOM with Redux store and Browser Router providers.
- **AppRoutes.jsx**: Configures routes and handles initial subscription status verification against the license API.
- **ProtectedRoute.jsx**: Prevents unauthorized path traversal using role-based parameters (e.g., admin, superadmin, security, host).
- **app/apiClient.js**: Configures an Axios instance pointing to the backend API (port 5000) and automatically injects JWT tokens from sessionStorage.

2.1 Front-end Pages (*src/pages/*)

- **LandingPage**: A kiosk-like homepage welcoming visitors and providing easy entry points to register.
- **VisitorRegistration**: Dynamic step-by-step wizard capturing visitor details, photograph, vehicle parameters, and host contact.
- **SecurityDashboard**: Terminal screen for guards to check visitors in/out, register walk-ins, print badges, and log vehicle movements.
- **HostDashboard**: Inbox panel for employees to invite visitors, approve/deny incoming gate requests, and manage personal schedules.
- **AdminDashboard**: Admin panel to manage company information, configure sites/departments, add hosts, review global logs, and adjust settings.
- **SuperAdminDashboard**: Master view containing billing logs, active sites, global logs, and license expiration tables.
- **LicenseRestricted**: Graceful fallback screen prompting administrators to activate or renew keys when license checks fail.

3. Backend Architecture & Usability

The backend is built with **Flask**, leveraging raw MySQL connections for performance and flexible query operations.

- **app.py**: Entrypoint setting up CORS, rate limiting, and blueprints. Features a global request filter to check license status and JWT validity.
- **db.py**: Establishes and manages connections to the MySQL database.
- **routes/auth_routes.py**: Handles user registration, login API, hashed password validations, and JWT issuance.
- **routes/visitor_routes.py**: Exposes paths for visitor registration, vehicle lookup, and real-time status.
- **routes/request_routes.py**: Manages host notifications, approval requests, and checkout status flags.
- **routes/security_routes.py**: Serves gate actions, records logs, processes check-ins/outs, and validates badges.
- **routes/admin_routes.py**: Handles site settings, logo uploading, user profile updates, and configurations.
- **routes/reports_routes.py**: Queries database logs to compile and export downloadable Excel/PDF reports.

4. Data Flow & Connectivity

- 1. Client Action:** A visitor registers at a kiosk or a guard processes check-out on the security dashboard.
- 2. Security & Header Injection:** The frontend Axios client retrieves the active JWT token from sessionStorage and attaches it to request headers.
- 3. Backend Middleware & Verification:** app.py intercepts the request to verify system license status and token signatures before routing.
- 4. DB Persistence:** The router delegates execution to the MySQL database via raw SQL bindings (db.py) and returns the JSON payload back to the client.